

Architectural Foundations of Deterministic Agent Memory: Mitigating Context Rot and Indirect Prompt Injections via Harvard-Decoupled State Meshes

Memory Mesh Protocol (MMP) and Memory Mesh Bridge (MMB): a deterministic long-term state architecture for autonomous AI agents.

Justin Paul Urbaniak

Architectural preview · September 2026 · Conceptual level only: no implementation details, schemas or algorithms

Abstract

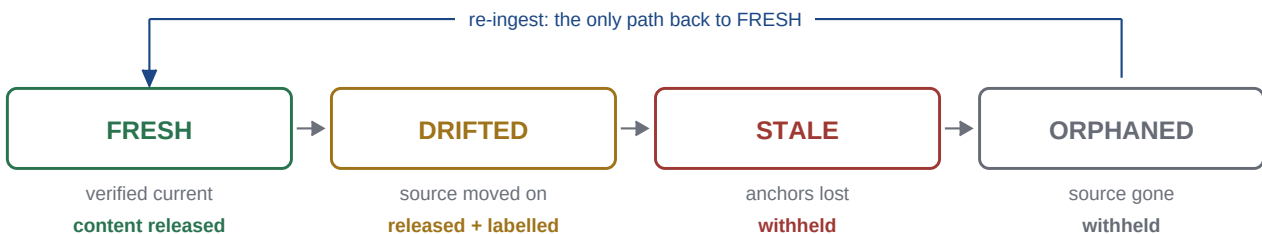
Autonomous agents keep their working state in the conversation transcript. The transcript is simultaneously the plan, the memory and the audit log, and it fails in all three roles at enterprise scale: it is lost when the context window ends, it grows more expensive with every step, and it cannot be audited afterwards. It also carries a structural security defect, in that retrieved material and operator instructions arrive through the same undifferentiated channel, so anything the agent reads can, in principle, instruct it.

This paper describes an architecture that removes state from the transcript entirely. Knowledge is held as a verified mesh in which every node maintains an independent link to its physical source and carries a verification verdict that is re-derived on every read. Execution state (plan position, step outcomes, and an append-only record of what happened and why) is held outside the context window, so a run may lose its context at any point and resume. The two are deliberately not merged: how knowledge evolved and what a given run did are different questions and are never answered from the same record.

The architecture is Harvard-decoupled. Material extracted from indexed sources is data by construction and can never acquire executable authority, regardless of its content. We argue that this is the only structurally sound answer to indirect prompt injection in autonomous CI/CD contexts, because it removes the need to recognize hostile instructions rather than attempting to filter them.

The State Lifecycle

A node is not either valid or invalid. Collapsing verification into a boolean discards the distinction between material that has moved on and material that has become unusable, and that distinction is what determines whether work can continue. The lifecycle is therefore four-valued.



Verification compares a node against its physical source and produces exactly one of these verdicts. FRESH means the source is unchanged. DRIFTED means the source changed but every anchored region was located again intact, so the agent may proceed and receives the material explicitly labelled as the extraction-time state together with a re-ingest recommendation. STALE means the source changed and the anchored material could not be located again. ORPHANED means the source no longer exists.

The decisive property is not the classification but what follows from it. STALE and ORPHANED nodes release no content: not filtered content, not a warning wrapped around stale content, but none. There is no override parameter and no degraded mode, by deliberate design: an emergency exit that exists will eventually be used, and the entire guarantee rests on its absence. Re-ingestion is the only route back to FRESH.

A run that halts costs time. A run that proceeds on demonstrably outdated material produces wrong artifacts, which then enter the mesh as new truth. The second failure is the one that cannot be undone, so the system blocks rather than warns.

Security and the Harvard Architecture

Retrieval-augmented agents share a vulnerability that better prompting does not address: the material an agent retrieves and the instructions an agent is given arrive in the same channel. Anyone able to influence an indexed artifact (a dependency README, a code comment, an issue description, a docstring) is writing into the instruction stream of every agent that later retrieves it. In an autonomous CI/CD pipeline, where the agent holds commit or deployment authority, this is a remote code execution path with a content-management interface.

MMP separates the two paths architecturally rather than filtering at the boundary:

- **Origin is structural.** Every node carries an origin classification. Material with origin 'extracted', meaning anything harvested from an indexed source, is data, permanently and by construction. It can be read, reasoned over and cited. It cannot instruct.
- **Authority is explicit and verified.** Executable authority is reserved to deliberately authored material, and is additionally conditioned on current verification state. An instruction that cannot be verified as current is not executed.
- **Execution capability stays closed.** The 'procedure' capability, a node carrying executable behaviour, remains closed in the current protocol generation. This is sequencing, not oversight: the separation is proven load-bearing before anything is permitted to execute.

The consequence is that an injected instruction placed in an indexed file lands in the data path. It is visible, attributable to a specific source, and inert. Filtering approaches must recognize hostile instructions in order to stop them, which makes their effectiveness a function of the attacker's creativity. Denying an entire class of material the authority to instruct does not require recognition at all.

Empirical Impact: Measured, Not Asserted

Externalized state makes it structurally possible to discard an agent's context between steps: the thread of work survives, because it was never in the transcript. Whether discarding is worth doing is a separate, arithmetic question, and it belongs to the harness and the workload, not to the state layer. What follows measures that operating strategy, not the protocol, and it does not depend on the protocol being present: any system that can externalize its execution state faces the same arithmetic.

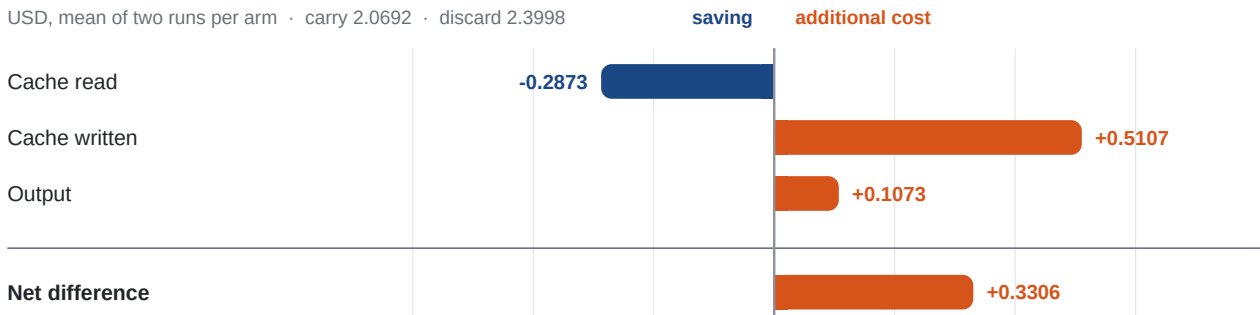
Two arms, the same task, files, model and machine: one context carrying all six steps of a plan, against six fresh contexts handing off a small result file on disk - a property of this test rig, not a constraint of the state layer. Two repetitions per arm in mirrored order, 14 isolated agent runs, 103 model requests, a corpus of roughly 11,000 tokens. Two is below the three repetitions per arm our own protocol required, so this run is a sign check and not a test: the design cannot produce a statistically significant result at all. Billing was reconstructed per request, every reading recomputed under three cache-warmth assumptions applied identically to both arms, and three independent reviews run against the result. The first version of that warmth normalisation applied the assumptions inconsistently; the arithmetic review caught it before publication, and correcting it moved the margin against the discard arm, not for it.

On total cost the discard arm was more expensive in every run, with no overlap between the arms under any of the three warmth readings; across the same four runs the margin spans +9.9 to +89.1 percent depending on how restart warmth is priced, the expensive end computed from the single cold start observed rather than run. We decline to reduce this to one number: the spread within the discard arm was 48 percent of the measured difference between the arms. What the run supports is the sign, under stated conditions:

On a carried payload of roughly 11,000 tokens, with steps seconds apart, in a general-purpose agent harness with the cache layout it shipped with in September 2026, discarding context between steps cost more than carrying it, and the work was indistinguishable.

Where the cost difference came from

USD, mean of two runs per arm · carry 2.0692 · discard 2.3998



The discard arm did save where it was supposed to: it read 0.2873 USD less out of cache. It paid that back more than twice over, because five additional preambles are written into the cache on restart instead of being read from it, at 12.5 times the price. Roughly 82 percent of that written block is unchanged standing text.

Every clause in that sentence is load-bearing. Change the payload, the spacing, or the harness, and the sign is not guaranteed to survive. That matters beyond the laboratory: the operating case this architecture exists for - a plan whose steps may be hours apart - is structurally the cold case, the expensive end of the range, and the case no run with steps seconds apart exercises at all.

On quality we could measure no difference at $n=2$. Wall-clock time was measured and is not reportable: the spread within a single arm was about six times the difference between the arms.

The four runs, individually

total cost per run, USD



The arms do not overlap: every discard run cost more than every carry run. The same picture shows why no percentage is quoted - the spread within the discard arm alone is 48 percent of the difference between the arms. Both within-arm gaps have identified contributors rather than being unexplained: a1 was the only cold start observed, which accounts for roughly 0.21 USD of its value, and b1 made 28 model requests where the other three runs made 25.

The run falsified three internals of our own pre-run cost model; those corrections are its most useful output.

- **One: carrying cost 2.96 times what we predicted,** because both requests per step and the volume carried were underestimated; the warm break-even threshold falls from roughly 108,000 carried tokens to roughly 36,500, which moves ordinary engineering tasks from below the line to sitting on it.
- **Two: most of the penalty was the harness, not the strategy.** Roughly 82 percent of what each warm restart cost in this run was an unchanged block of standing text that the harness re-writes into the cache on every restart instead of reading it, at 12.5 times the price, as measured in September 2026. Priced as a read, the discard arm would have won this run, by a margin inside the run-to-run spread we decline to quote - which moves the answer from a clear loss to a coin toss. That is a property of a particular execution harness and a particular cache layout at a particular date. It is not a property of discarding context, and it is not a property of the protocol.
- **Three: chain length does not cancel out.** At fixed step size the balance is quadratic against carrying, and with warm restarts discarding wins from roughly nine steps of 5,000 tokens each; cold restarts move that crossover out into the low twenties.

Break-even is therefore a surface, not a threshold, moving along at least three axes: payload carried per step, length of the chain, and how the executing harness partitions its cache, which dominated here. A fourth term belongs in the same balance and is missing from every figure above: the cost of carrying results between steps in files, which a discarding arm cannot avoid and which we can size only by estimate. Cost is also not an energy measure - the discard arm processed fewer tokens and paid more money, because the tokens moved from a cheap price class to an expensive one, and we did not measure energy. And where an input rate limit binds rather than a budget, the sign of the relevant quantity is the opposite of the sign of the cost: in this run the carry arm needed about 1.3 times the input-token throughput for the same work.

By our own pre-registered criteria, this run should have been discarded: we required three repetitions per arm and ran two, and named 12 model requests per step as the point below which the expected result flips, where we measured 4.17. Nothing here is therefore offered as a statistically significant result. Nor does the run locate the surface: its corpus sits about a factor of three below even the corrected threshold, so it confirms that a prediction lands on the expected side of the curve without locating the curve, and a larger corpus prepared for exactly this question was not the one we ran. We report it because the corrections survive the sample size even where the percentages do not. The full write-up accompanies this preview as a separate document; the cost model and the review record are held in the project archive; the decisive run - a corpus at the threshold, three repetitions per arm, cold restarts - accompanies the protocol specification.

Token reduction is never the sole objective. The objective is the equilibrium of reduced compute and undiminished output quality: no destructive truncation, no summarization that discards detail a later query will need.

Roadmap and Commercial Program

Verified state with source anchoring and a fail-closed read path is implemented, as is externalized execution state with resumption after context loss. The first empirical run and the three corrections it forced on our own cost model are published above; the decisive run is open. A public protocol specification is in preparation and will describe contracts, state semantics and guarantees at a depth sufficient for independent implementation, without publishing the reference engine.

STAGE	SCOPE	STATUS
Verified state and anchoring	Source-linked nodes, four-valued lifecycle, fail-closed read path	Implemented
Externalized execution state	Plans, step lifecycle, append-only execution record, resumption after context loss	Implemented
First empirical run	Sign check at 11,000 carried tokens; three corrections to our own cost model	Published here
Decisive empirical run	Corpus at the threshold, three repetitions per arm, cold restarts	Open
Public protocol specification	Contracts and state semantics at implementation depth	In preparation
Closed B2B pilot	Selected industry partners, on-premise deployment	Accepting enquiries

The system is architected for single-tenant, on-premise operation. This is a design commitment rather than a deployment option: verification against a physical source tree, and an audit record an operator can actually hold, both presuppose that the operator controls the machine.

A closed pilot program is open to a small number of selected industry partners, prioritising organisations running autonomous agents against large proprietary code or document corpora under audit obligations. The Enterprise Engine and On-Premise Core are offered under BSL 1.1 or commercial licence terms.

Enquiries: see the contact section of the accompanying repository README.

This preview communicates architecture and behaviour only. Internal state schemas, verification and relocation algorithms, and implementation source are not part of this publication. © 2026, all rights reserved.